This version calls stub to make calls to connect or disconnect be handled by the stubbing functionality to return true rather than making the actual method call.

Stubs don't get verified, so they are most useful for replacing an expensive or flaky method call with a canned value for use as part of larger logic.

If you want more complex mock object behavior, the longstanding Ruby library Mocha[1] is the next step up in using mock objects in minitest.

Organizing and Running Tests

The test cases we've seen so far are all runnable Ruby programs. If, for example, the test case for the Roman class was in a file called test_roman.rb, we could run the tests from the command line using this:

```
$ ruby test_roman.rb
Run options: --seed 35590
# Running:
..
Finished in 0.000379s, 5277.0448 runs/s, 18469.6568 assertions/s.
2 runs, 7 assertions, 0 failures, 0 errors, 0 skips
```

Minitest is clever enough to run the tests even though there's no main program. It collects all the test case classes and runs each in turn.

If we want, we can ask it to run a particular set of test methods based on a naming pattern:

```
$ ruby test_roman.rb -n test_range
Run options: -n test_range --seed 46631
# Running:
.
Finished in 0.000413s, 2421.3075 runs/s, 4842.6150 assertions/s.
1 runs, 2 assertions, 0 failures, 0 errors, 0 skips
```

In this case, minitest will run test methods whose names exactly match the text passed to -n. That's pretty restrictive, so if you want to run more than one test based on a naming pattern, if you include any regular expression punctuation in the argument, minitest will match the test name against the regular expression:

```
$ ruby test_roman.rb -n /range/
Run options: -n /range/ --seed 10205
# Running:
.
Finished in 0.000605s, 1652.8926 runs/s, 3305.7852 assertions/s.
1 runs, 2 assertions, 0 failures, 0 errors, 0 skips
```

This last capability is a great way of grouping your tests. Use meaningful names, and you'll be able to run (for example) all the shopping-cart-related tests by running tests with -n /cart/.

Where to Put Tests

Once you get into unit testing, you may well find yourself generating as much test code as production code. All of those tests have to live somewhere. The problem is that if you put

1. https://mocha.jamesmead.org